

1. Dataset Source

The dataset used for this experiment is the **Iris Dataset**, which is widely used for classification tasks in machine learning.

Dataset Link:

<https://www.kaggle.com/datasets/uciml/iris>

The dataset contains measurements of iris flowers and is used to classify them into different species.

2. Dataset Description

The **Iris Dataset** consists of **150 observations** of iris flowers belonging to **three species**.

Features (Independent Variables)

Feature	Description
Sepal Length	Length of the sepal (cm)
Sepal Width	Width of the sepal (cm)
Petal Length	Length of the petal (cm)
Petal Width	Width of the petal (cm)

Target Variable

Variable	Description
Species	Type of iris flower (Setosa, Versicolor, Virginica)

Dataset Characteristics

- Total Instances: **150**
- Total Features: **4**

- Target Classes: **3**
- Type: **Supervised Classification Dataset**

The dataset is balanced and contains equal distribution of the three flower species.

3. Mathematical Formulation of the Algorithm

Decision Tree

A **Decision Tree** is a supervised learning algorithm that splits the dataset into subsets based on feature values.

The best split is selected using **Entropy** or **Gini Index**.

Entropy Formula

$$Entropy(S) = -\sum_{i=1}^n p_i \log_2(p_i) \quad Entropy(S) = -\sum_{i=1}^n p_i \log_2(p_i)$$

Where:

- p_i = probability of class i

Information Gain

$$IG = Entropy(parent) - \sum |child| |parent| \times Entropy(child) \quad IG = Entropy(parent) - \sum |parent| |child| \times Entropy(child)$$

The feature with **maximum information gain** is selected for splitting.

Random Forest

A **Random Forest** is an ensemble method that builds **multiple decision trees** and combines their outputs.

Prediction formula:

$$Prediction = Majority\ Voting(T_1, T_2, T_3, \dots, T_n) \quad Prediction = Majority\ Voting(T_1, T_2, T_3, \dots, T_n)$$

Where:

- $T_n T_n$ = individual decision trees

This improves **accuracy and reduces overfitting**.

4. Algorithm Limitations

Decision Tree Limitations

- Prone to **overfitting**, especially with small datasets
- Small changes in data may produce **different tree structures**
- Not very stable for complex datasets

Random Forest Limitations

- Requires **more computational resources**
 - Harder to interpret compared to decision trees
 - Training time is higher because many trees are built
-

5. Methodology / Workflow

The experiment follows these steps:

Step 1: Data Collection

The **Iris Dataset** is collected from Kaggle.

Step 2: Data Preprocessing

- Load dataset using Pandas
- Check for missing values
- Separate features and target variable

Step 3: Train-Test Split

Dataset is divided into:

- **80% Training Data**
- **20% Testing Data**

Step 4: Model Training

Two models are trained:

- **Decision Tree**
- **Random Forest**

Step 5: Model Prediction

Both models predict the species of flowers using test data.

Step 6: Model Evaluation

Performance is evaluated using:

- Accuracy
 - Confusion Matrix
 - Classification Report
-

6. Performance Analysis

The performance of both models is analyzed using evaluation metrics.

Accuracy

Accuracy measures the percentage of correctly predicted samples.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Confusion Matrix

The confusion matrix compares **actual vs predicted classes**.

Example results:

Model	Accuracy
Decision Tree	1.00
Random Forest	1.00

Both models performed well on the **Iris Dataset** due to its simple and well-separated features.

Random Forest is generally considered **more robust** because it uses multiple trees.

7. Hyperparameter Tuning

Hyperparameter tuning improves the performance of machine learning models by adjusting parameters.

Decision Tree Parameters

- **max_depth** – Maximum depth of tree
- **min_samples_split** – Minimum samples required to split a node
- **criterion** – Split quality measure (gini or entropy)

Example:

```
DecisionTreeClassifier(max_depth=3)
```

Random Forest Parameters

- **n_estimators** – Number of trees
- **max_depth** – Maximum tree depth
- **min_samples_split**

Example:

```
RandomForestClassifier(n_estimators=100, max_depth=5)
```

Tuning these parameters helps reduce **overfitting** and improves model accuracy.

Conclusion

In this experiment, classification was performed using **Decision Tree** and **Random Forest** on the **Iris Dataset**.

The dataset was preprocessed and divided into training and testing sets. Both models were trained to classify iris flowers based on features such as sepal length, sepal width, petal length, and petal width. The performance of the models was evaluated using accuracy score, confusion matrix, and classification report.

The results showed that both models achieved **100% accuracy**, indicating that all test samples were correctly classified. While the Decision Tree model is simple and easy to interpret, the Random Forest model is generally more robust because it combines multiple decision trees to improve prediction reliability and reduce overfitting.